Sunol FlowLab Comprehensive Batched Implementation Plan

1. Executive assessment

Sunol FlowLab has a strong foundation:

- Deterministic fixed-step simulation.
- A two-pass DAG flow solver.
- A single storage-volume mutation path.
- Mass-balance and replay invariants.
- Clean simulation/presentation separation.
- Data-driven plant configurations.
- A functional Phase 3 headworks presentation.

The correct immediate strategy is **closure before expansion**. Phase 3 is implemented but its exit gate remains open, WP4.1 still requires closure verification, and Phase 4a is explicitly blocked by WP4.7. The roadmap also prohibits beginning another broad subsystem while audit closure remains open.

The roadmap's WP4.2–WP4.7 sequence should remain the backbone, but the repository review exposed three additional matters that need to be incorporated:

1. **A typed hydraulic-head contract is missing.** Gravity flow currently discovers hydraulic head through `has_method()` and property probing, contrary to the repository's own guardrail against duck-typed dispatch.
2. **The controller model is being described inconsistently.** The implementation accumulates `gain × error` into the previous output and adds `kp × change-in-error`; the active Phase 3 configuration uses both `gain = 1.5` and `kp = 20.0`. This is not the "P-only controller" described by the WP4.1b handoff.
3. **Phase 4 will trigger the first justified generalization of basin-specific service and presentation code.** The existing service command and panel are generic in behavior but basin-specific in name and configuration. That refactor should happen only when filters become the second concrete user.

---

2. Governing implementation rules

These rules apply to every batch:

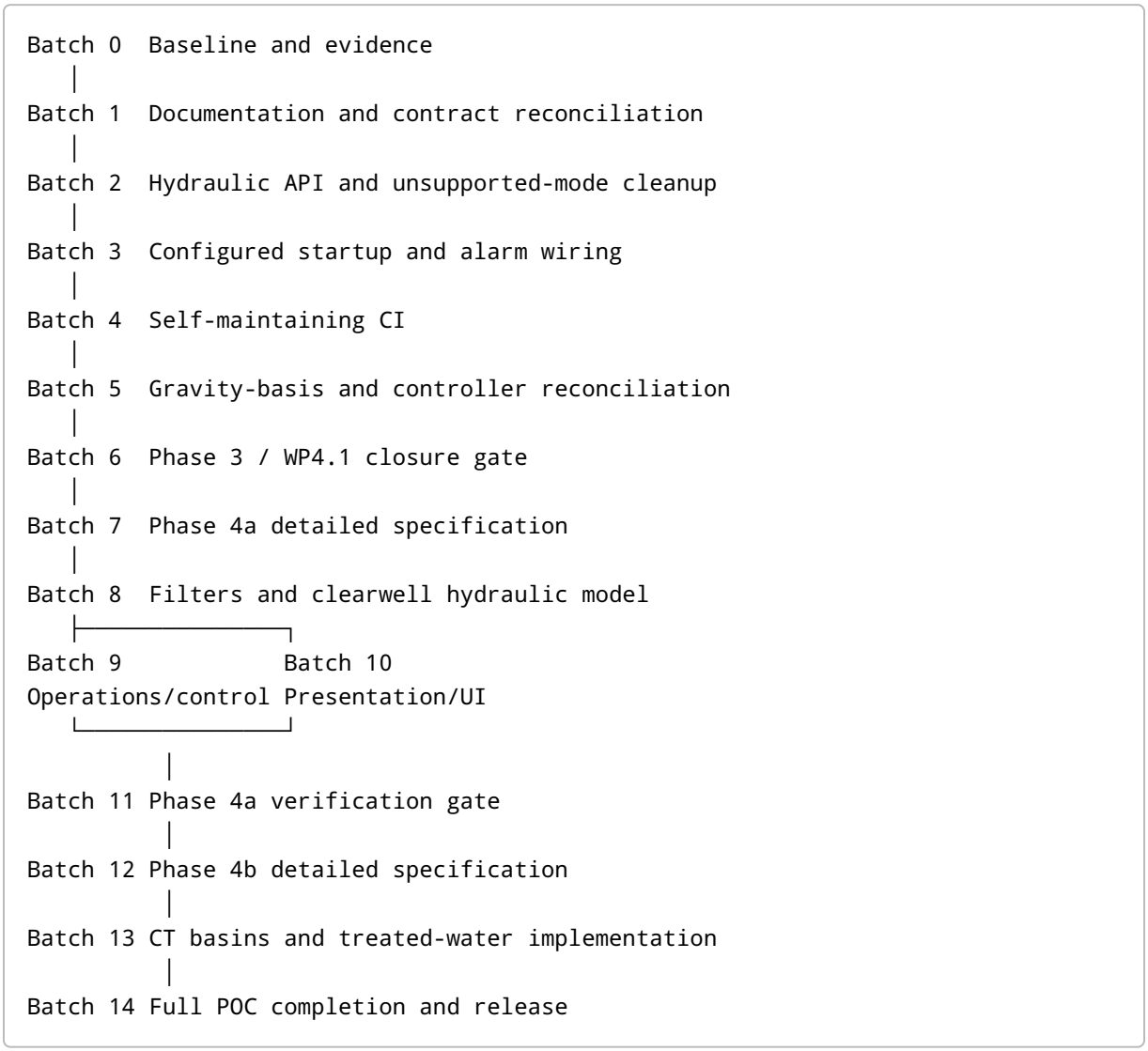
1. One batch should produce one reviewable commit or small PR.
2. Every behavior change must include a production-path regression test.
3. Earlier plant configurations remain immutable regression fixtures.
4. Phase 4 receives a new plant configuration rather than modifying `phase3_headworks`.

5. New configuration fields require schema, semantic-validator, fixture, and documentation updates in the same batch.
6. No test may reimplement the solver, tick pipeline, balance logic, or controller.
7. No batch may claim success without actual collected/passed/failed test totals.
8. Work ends with a clean tree and intact final file lines.
9. Documentation describes executable reality, not proposed future architecture.
10. Features are added only when required by the next playable milestone.

These follow the repository's existing agent guardrails and roadmap philosophy.

---

3. Critical path



---

4. Batch status summary

Batch	Work	Present status	Blocked by	Parallel potential
0	Baseline and evidence	Ready	Test environment	No
1	Documentation truth pass	Ready	Batch 0 evidence	Human data gathering
2	Hydraulic contract cleanup	Ready	Batch 1 contract decisions	Limited
3	Startup and alarms	Ready after Batch 2	Stable production contracts	Human data gathering
4	CI self-discovery	Ready after Batch 3	Final audit test set	Human data gathering
5	Gravity and controller reconciliation	Partially ready	Real plant data for calibrated elevations	Analysis and data collection can run together
6	Phase 3 closure	Blocked	Batches 0–5	No
7	Phase 4a detailed plan	Blocked	Batch 6	Non-binding source-data research may start earlier
8	Filter/clearwell hydraulics	Blocked	Batch 7 and design basis	Tests and presentation-map preparation after IDs freeze
9	Phase 4a operations/ control	Blocked	Batch 8	Can run beside Batch 10
10	Phase 4a presentation/UI	Blocked	Batch 8	Can run beside Batch 9
11	Phase 4a closure	Blocked	Batches 9–10	No
12	Phase 4b plan	Blocked	Batch 11	Data gathering only
13	Phase 4b implementation	Blocked	Batch 12	Control and presentation substreams
14	POC closure/release	Blocked	Batch 13	No

5. Detailed implementation batches

Batch 0 — Establish a trustworthy baseline

Goal

Prove the actual state of `main` before modifying it.

Work

1. Confirm the worktree is clean and no files are truncated.
2. Import the project using Godot 4.7.
3. Run the complete GUT suite.
4. Run configuration-schema validation.
5. Launch the configured main scene: `res://scenes/plant/headworks_area.tscn`.
6. Run the Phase 3 headless and visual smoke tests.
7. Capture:
8. Test scripts collected.
9. Individual tests executed.
10. Passed, failed, skipped, and pending counts.
11. Configurations validated.
12. Exact commit SHA.
13. CI run URL.
14. Record hashes or diffs for the Phase 1 and Phase 2 configurations and tests. These become the "must remain unchanged" comparison set.
15. Record current Phase 3 steady-state values without treating them as newly approved goldens.

The project configuration identifies Godot 4.7 and the headworks scene as the current runtime, while the README still advertises Godot 4.5 and different launch scenes.

Exit criteria

- Nonzero tests collected.
- Zero failing tests.
- Configuration validation passes.
- Main scene launches.
- Evidence is committed or linked from a gate-evidence document.
- Worktree is clean.

Hard blocker

When tests cannot be executed, use the repository's required wording:

> Tests written but NOT executed — unverified.

No Phase 4 work begins from an unverified baseline.

Batch 1 — Reconcile documentation with executable reality

Goal

Make active documentation accurately describe the current repository and remove speculative architecture from binding contracts.

Work

README

- Change Godot version from 4.5 to 4.7.
- Identify `headworks_area.tscn` as the main scene.
- Correct the current process train.
- Replace the static “55 passed” badge with a workflow-status badge or remove the hardcoded count.
- Verify clean-clone run instructions.

Documentation authority

- Resolve the tension between `INDEX.md`, `AGENTS.md`, and `REPOSITORY_ARCHITECTURE.md`.
- Keep the architecture document authoritative for structural boundaries.
- Make schema descriptions authoritative for configuration fields.
- Make the roadmap authoritative for work sequencing and current status.

Repository architecture

Reduce `REPOSITORY_ARCHITECTURE.md` to:

- Current directories that actually exist.
- Dependency direction.
- Fixed tick order.
- Command/event/snapshot boundaries.
- Configuration loading and validation.
- Protected invariants.
- Current extension points.

Move the large speculative future folder tree, telemetry system, scenario framework, state-machine catalog, unused process-unit classes, and future service list into an explicitly non-binding appendix or archive.

The current architecture document presents many planned modules as if they are part of the active structure, including filter state machines, telemetry, scenarios, multiple controllers, process-specific models, and future scenes.

Process contracts

Rewrite `PROCESS_UNIT_CONTRACTS.md` around production symbols:

- `unit_id`, not `id`.
- `operating_state`, not `state`.
- Actual `StorageUnit` methods and fields.
- Actual `FlowLink` modes.
- Actual `ThresholdAlarm` fields.
- Actual controller configuration and mathematics.
- Implemented events only.
- Implemented commands only.

Delete or explicitly defer:

- `COMMANDED`.
- Reverse flow.
- `flow_coefficient`.
- `fail_state`.
- Generic alarm priorities that do not exist.
- Controller methods that do not exist.
- State-machine behavior not implemented.
- Instrument classes not implemented.
- Operator percentage splitting as a separate algorithm.

Topology and control documents

- Update the design-basis text to reflect Phase 3 gravity flow.
- Remove the obsolete statement that the applied-channel demand is non-self-regulating.
- Reconcile the distribution-box description with FlowSolver proration.
- Correct the stale reference to `default_surface_water_plant`.
- Reconcile the controller description with the actual velocity-form implementation.
- Correct the Phase 3 storage-unit count wherever it is inconsistent.

Guides and tests

- Update testing terminology from “storage nodes” to `StorageUnit`.
- Remove future lead-lag, splitter-controller, and scenario-regression promises from the active testing contract unless tests exist.
- Validate all active documentation links.

Exit criteria

- Every active file path exists.
- Every active public method exists.
- Every documented configuration field exists in its schema and production loader.
- Speculative material is clearly non-binding.
- Documentation link validation passes.

- No runtime behavior changes in this batch.

Batch 2 — Make the hydraulic contract explicit and remove unsupported paths

This batch expands WP4.3 and WP4.4 with one required contract repair.

Batch 2.1 — Typed hydraulic-head interface

Problem

`FlowLink` obtains hydraulic head through:

- `has_method("water_surface_elevation_m")`.
- A property-membership test for `reference_head_m`.

`FlowPort` also carries both `parent_unit` and `owner_unit` as loosely typed references.

Work

1. Define one typed hydraulic-head method on the process-unit contract, such as: `get_hydraulic_head_m() -> float`.
2. Implement it on:
3. `StorageUnit`.
4. `ExternalBoundary`.
5. Type the canonical FlowPort unit reference as `ProcessUnit`.
6. Choose one canonical unit-reference field.
7. Migrate production callers.
8. Remove the duplicate field only after a repository-wide reference check.
9. Make unsupported unit types fail during validation/build rather than silently returning zero head.
10. Remove duck-typed head discovery from `FlowLink`.

Tests

- Storage head equals floor elevation plus level.
- Boundary head equals reference head.
- Gravity link between valid typed endpoints.
- Invalid endpoint type rejected.
- Replay unchanged.
- Phase 1 and Phase 2 unchanged.

Batch 2.2 — Remove `COMMANDED`

Work

Remove `COMMANDED` from:

- Topology schema.
- Plant validator.
- `FlowLink`.
- Tests and fixtures.
- Simulation rules.
- Process contracts.
- Configuration reference.
- Examples and archived-active cross references.

Do not implement it.

The current production path warns once and treats `COMMANDED` as fully open `RESTRICTED`, while the schema advertises it as supported.

Exit criteria

- Invalid `COMMANDED` configuration fails clearly.
- No production fallback remains.
- `RESTRICTED` and `GRAVITY` results remain unchanged.

Batch 2.3 — Remove reverse-flow support

Work

- Remove `reverse_flow_allowed`.
- Remove the negative-head reverse-flow branch.
- Define negative head as zero forward flow.
- Remove the field from snapshots, contracts, and tests.
- Preserve the directed-acyclic topology rule.
- Add positive, equal, and negative-head tests.

Sequencing

These three sub-batches touch the same hydraulic files and should not be implemented concurrently.

Batch 3 — Make startup, alarms, and presentation configuration-driven

Goal

Ensure the first visual and headless states come entirely from plant configuration.

Current gaps

- The headworks bootstrap hardcodes four startup valves at 80%.
- It loads the presentation map with a duplicate JSON helper.
- `ConfigLoader` loads and validates `alarms.json`, but `PlantFactory` and the bootstrap do not instantiate those alarms.
- The engine already owns and evaluates an `AlarmEngine`.

Work

1. Put all demonstration valve states in `initial_conditions.json`.
2. Delete `STARTUP_OPEN_VALVES` and `_queue_startup_commands()`.
3. Ensure the initial visual snapshot reflects configuration before the first simulation tick.
4. Replace `_load_json_dictionary()` with `PresentationMapHandler.load_map()`.
5. In the existing bootstrap path:
6. Iterate `config.alarms_data["alarms"]`.
7. Construct `ThresholdAlarm`.
8. Initialize it from configuration.
9. Register it with `host.engine.alarm_engine`.
10. Do not add an alarm registry, alarm service, dependency-injection framework, or new factory hierarchy.
11. Add a minimal alarm presentation:
12. Active alarm list, or
13. Unit alarm indicator driven by the snapshot.
14. Verify both configured Phase 3 alarm IDs appear in the snapshot.

Tests

- Configured initial valve positions appear in the first snapshot.
- No scene-local startup commands remain.
- High alarm activates after its configured delay.
- Low alarm activates.
- Deadband prevents chatter.
- Alarm clears once.

- One transition produces one event.
- Headless and visual startup states match.
- Five-basin outage demonstration still works.

Exit criteria

The default visual plant is configuration-driven and both configured alarms are visible and functional.

Batch 4 — Make CI self-maintaining

Goal

Remove the manually maintained test-script count without weakening defensive GUT checks.

Current gap

The workflow currently hardcodes `EXPECTED_SCRIPTS=33`, requiring every test addition or removal to modify CI manually.

Work

1. Derive the expected test-script count from: `tests/**/test_*.gd`.
2. Print:
3. Derived script count.
4. GUT-loaded script count.
5. Test totals.
6. Fail when:
7. Zero tests are collected.
8. A test script is skipped.
9. A parse or base-class error occurs.
10. Loaded and discovered script counts differ.
11. GUT reports failures.
12. Add workflow self-tests using temporary:
13. Valid test script.
14. Unparseable test script.
15. Ensure temporary files are always removed.
16. Preserve configuration-schema positive and negative checks.

17. Add lightweight documentation-link validation.
18. Add targeted repository checks confirming removed hydraulic terms do not return to active production files.

Exit criteria

Adding or removing a correctly named test requires no CI constant update, and malformed test files fail loudly.

Batch 5 — Reconcile gravity data and controller behavior

This batch has parallel analysis and data-gathering tracks.

Batch 5A — Controller-semantics audit

Critical finding

Do not add a new integral term based on the current WP4.1b description.

The current algorithm is:

```
output =
    previous_output
    + gain × error
    + kp × (error - previous_error)
    + kd × second_error_difference
```

Because `gain × error` is accumulated into `previous_output`, `gain` already acts as an integral-like term per simulation tick. The active configuration also uses a nonzero `kp`.

Work

1. Create focused controller characterization tests:

2. Constant error.

3. Step disturbance.
4. Deadband disabled.
5. Deadband enabled.
6. Output unsaturated.
7. Output saturated.
8. Bumpless MANUAL-to-AUTO transfer.

9. Measure whether the apparent Phase 3 offset is caused by:

10. Deadband.

11. A limit cycle and the averaging window.
12. Valve saturation.
13. Gravity equalization.
14. Insufficient simulation duration.
15. Controller interaction.

16. Define and document the coefficient mapping:

17. `gain`: integral increment per tick.

18. `kp`: proportional term in velocity form.
19. `kd`: derivative term in velocity form.
20. Decide whether coefficient names remain for backward compatibility.
21. Prefer Phase 3 configuration retuning over a shared algorithm change.
22. Any shared controller algorithm change becomes a separate work package with defaults that preserve all prior plant results.

Owner decision after evidence

Choose one:

- Accept the demonstrated limit-cycle/deadband behavior.
- Retune Phase 3 `gain`, `kp`, deadband, or setpoint.
- Approve a separate backward-compatible controller-algorithm change.

The decision must follow characterization, not precede it.

Batch 5B — Real gravity-basis reconciliation

Data needed

- Vertical datum.
- Eleven Phase 3 storage floor elevations.
- Normal operating water-surface elevations.
- Four boundary heads.
- Design head or conveyance information for 24 links.
- Confirmation of any pumped links.
- Plan coordinates and footprints for later visual accuracy.

The repository already contains a detailed human-data checklist.

Work when data is available

1. Replace provisional floor elevations.
2. Replace boundary heads.
3. Recalculate design heads.

4. Preserve level and alarm fields as relative depths.
5. Confirm `max_flow_m3s` capacities against source documentation.
6. Re-align initial volumes.
7. Rebaseline Phase 3 expected values from documented head calculations.
8. Cite source drawings or manuals in the design-basis note.
9. Recheck visual positions separately from hydraulics.

Blocker classification

Missing real elevations are a **soft blocker for calibrated realism**, not necessarily a hard blocker for Phase 3 functional closure. The existing WP4.1b dispatch explicitly permits leaving the provisional cascade in place when the real values are unavailable, provided the omission is documented.

Batch 6 — Close Phase 3 and WP4.1

Goal

Independently certify the headworks milestone before authoring active Phase 4 architecture.

Verification matrix

1. Full GUT suite.
2. Full config-schema suite.
3. Deterministic replay.
4. Registration-order permutation.
5. Mass-balance tolerance unchanged.
6. No negative storage.
7. 100,000-tick soak.
8. Demand and service-state churn.
9. Five-basin outage.
10. Basin return-to-service.
11. Configured startup.
12. High and low alarms.
13. Alarm event uniqueness.
14. Headless/visual parity.
15. Phase 1 and Phase 2 unchanged.
16. Main-scene visual smoke test.
17. Clean tree.
18. Green `main` CI.

Documentation changes

- Mark Phase 3 delivered.
- Mark WP4.1 delivered or explicitly provisional-calibrated.
- Record exact test totals and CI evidence.
- Close or supersede WP4.1 dispatch documents.

- Identify unresolved plant-data items without leaving the gate ambiguously open.

Gate

Phase 4a remains blocked until this batch passes.

---

6. Parallel human-data tracks

These tracks may begin before Phase 3 closes, but their output remains non-binding until the appropriate implementation gate opens.

Human Track H1 — Phase 3 hydraulic basis

Use the existing WP4.1b checklist.

Human Track H2 — Phase 4a filter and clearwell basis

Gather:

- Applied-channel normal operating elevation.
- Filter influent and effluent elevations.
- Per-filter design/rated flow.
- Filter footprint and effective storage volume.
- Minimum operating depth.
- High and spill levels.
- Clearwell floor, capacity, surface area, and operating level.
- Clearwell downstream boundary head.
- Clearwell design demand.
- Filter isolation semantics.
- Which valves are common and which are individual.
- Desired clearwell level setpoint.
- Site-plan positions for twelve filters and the clearwell.

Store this as a source-data worksheet or non-binding research note. Do not update active topology contracts before Batch 7.

Human Track H3 — Phase 4b basis

Gather:

- CT basin floor elevations and volumes.
- Clearwell-to-CT conveyance capacities.
- CT-to-treated-reservoir capacities.
- Treated-reservoir floor, capacity, and normal level.

- System-demand design flow.
- Parallel CT split expectations.
- Required supervisory-control intent.
- High/low alarm thresholds.

---

7. Phase 4a — Filters and clearwell

The roadmap limits Phase 4a to twelve filters, existing-solver distribution, clearwell storage, minimum operability control, service state, snapshot-driven visuals, and verification. Backwash and filter-to-waste remain excluded.

Batch 7 — Author the detailed Phase 4a specification

Goal

Resolve all Phase 4a design decisions before code or active configuration changes.

Required decisions

1. Create a new plant: `config/plants/phase4a_filtration/`.
2. Preserve `phase3_headworks` unchanged.
3. Extend the Phase 3 topology rather than mutating its regression fixture.
4. Model each filter as a generic `StorageUnit`.
5. Do not introduce `FilterModel`, filter inheritance, filter state machines, or filter-media calculations.
6. Use FlowSolver proration for the twelve-way split.
7. Use GRAVITY links when the required head data is available.
8. Define clearwell storage and downstream demand.
9. Define filter service-state behavior.
10. Define the minimum control arrangement.
11. Define the exact alarm set.
12. Define the presentation IDs and positions.
13. Define work packages and file ownership before implementation.

Recommended minimum control design

Use one shared filter-bank feed actuator referenced by the twelve filter inlet links.

One existing `LevelController` reads clearwell level and commands this shared actuator:

- Clearwell low: increase filter-bank feed.
- Clearwell high: decrease filter-bank feed.
- Individual filter service commands disable only that filter’s links.
- FlowSolver redistributes flow among remaining filters.

This provides one operational loop without adding a splitter controller, cascade engine, supervisory framework, or twelve interacting level controllers.

Exit criteria

A detailed Phase 4a plan exists with topology, capacities, IDs, tests, configuration files, control pairing, presentation scope, and done-when criteria.

Batch 8 — Implement Phase 4a hydraulics and configuration

Goal

Create a headless twelve-filter and clearwell plant using existing domain classes.

Topology

```
Phase 3 Headworks
       │
APPLIED_CHANNEL_01
       │
       ├── FILTER_01 ──┐
       ├── FILTER_02 ──┤
       ├── ...         ├── CLEARWELL_01
       └── FILTER_12 ──┘
                       │
                DOWNSTREAM_DEMAND
```

Work

1. Extend the hydraulic design-basis table before creating capacities.
2. Create the Phase 4a plant folder:
3. `plant.json`
4. `topology.json`
5. `initial_conditions.json`
6. `controllers.json`
7. `alarms.json`
8. `presentation_map.json`
9. Replace the Phase 3 `FILTER_FEED_01` terminal boundary with:
10. Twelve filter `StorageUnit`s.

11. Twelve inlet paths.
12. Twelve outlet paths.
13. One clearwell `StorageUnit`.
14. One downstream boundary.
15. Add the shared filter-bank actuator.
16. Use existing service-state link enablement.
17. Add no filter-specific production class.
18. Add no new configuration fields unless a concrete failure proves one necessary.
19. Validate the complete configuration with schemas and semantic validation.

Hydraulic tests

- Twelve filters receive flow.
- Declaration order does not change the split.
- Total filter inflow equals applied-channel outflow.
- Total clearwell inflow equals filter outflow.
- One filter out of service carries zero flow.
- Remaining filters redistribute.
- Multiple unavailable filters respect remaining capacity.
- OUTLET low-level cutoffs work.
- DRAIN paths remain available when out of service.
- No negative storage.
- Mass balance remains within the standard tolerance.
- Replay remains exact.

Exit criteria

The Phase 4a plant runs headlessly with twelve filters and one clearwell, without new domain abstractions.

Batch 9 — Add Phase 4a operations, controls, alarms, and service generalization

Goal

Make the filter bank operable.

Work

1. Add the clearwell-level controller using the control design fixed in Batch 7.
2. Configure its initial mode, output, setpoint, and bumpless-transfer behavior.
3. Add clearwell high and low alarms.
4. Retain applied-channel alarms.
5. Verify insufficient filter capacity produces observable upstream or downstream level consequences.
6. Generalize service-state naming now that two concrete unit families exist:

7. Make `SetUnitServiceCommand` the canonical generic command.

8. Retain `SetBasinServiceCommand` as a compatibility alias where required.
9. Replace `BasinServicePanel` with a configurable `UnitServicePanel`.
10. Populate the panel from configured unit IDs or presentation metadata.
11. Do not add interlocks or permissives unless an actual unsafe transition cannot be represented by service state and link enablement.

Tests

- Filter out-of-service and return-to-service.
- Filter drain remains available.
- Shared filter-bank command affects all active filter inlet links.
- Clearwell controller direction is stable.
- Bumpless MANUAL-to-AUTO transfer.
- One unavailable filter does not alter other filter service states.
- Low clearwell alarm.
- High clearwell alarm.
- Controller replay.

Exit criteria

Operators can isolate and restore filters and the clearwell remains controllable under supported capacity conditions.

Batch 10 — Extend the presentation to filters and clearwell

Goal

Make Phase 4a visibly operable without introducing a second presentation framework.

Work

1. Reuse the current topology-plus-presentation-map adapter.
2. Generalize `HeadworksPresentationAdapter` only now that a second process section exists:

3. Rename it to a plant-level adapter, or

4. Move generic behavior into one plant adapter and preserve a compatibility wrapper.
5. Do not create presentation inheritance trees.

6. Add low-poly functional representations of:

7. Twelve filters.

8. Clearwell.
9. Filter links.
10. Downstream boundary.
11. Animate water levels from snapshots.
12. Animate filter-bank and individual valve states.
13. Show active alarms.
14. Allow every filter and the clearwell to be selected.
15. Add the generic service panel.
16. Use `PresentationMapHandler` for all placement loading.
17. Keep asset polish subordinate to function.

Parallelization

After Batch 8 freezes IDs and topology, Batch 10 can run alongside Batch 9 on a separate branch. Coordinate ownership of:

- Main scene.
- Presentation map.
- UI scene composition.

Merge Batch 9 before final parity testing.

Exit criteria

The visual plant shows the same states and flows as the headless snapshot.

Batch 11 — Close Phase 4a

Verification scenarios

1. Normal twelve-filter operation.
2. One filter out of service.
3. Several filters out of service while sufficient capacity remains.
4. Insufficient filter capacity.
5. Filter returned to service.
6. Filter drain-down.
7. Clearwell demand change.
8. Clearwell controller MANUAL/AUTO transfer.
9. Applied-channel high-level response.
10. Clearwell high and low alarms.
11. 100,000-tick full Phase 4a soak.
12. Service-state and demand churn.
13. Mass balance.
14. No negative storage.
15. Replay.
16. Registration-order permutation.

17. Headless/visual parity.
18. Selection and service UI smoke test.
19. Phase 1–3 regression suite unchanged.

Exit criteria

- Exact test totals recorded.
- CI green.
- Phase 4a marked delivered.
- Worktree clean.
- Phase 4b gate opens.

---

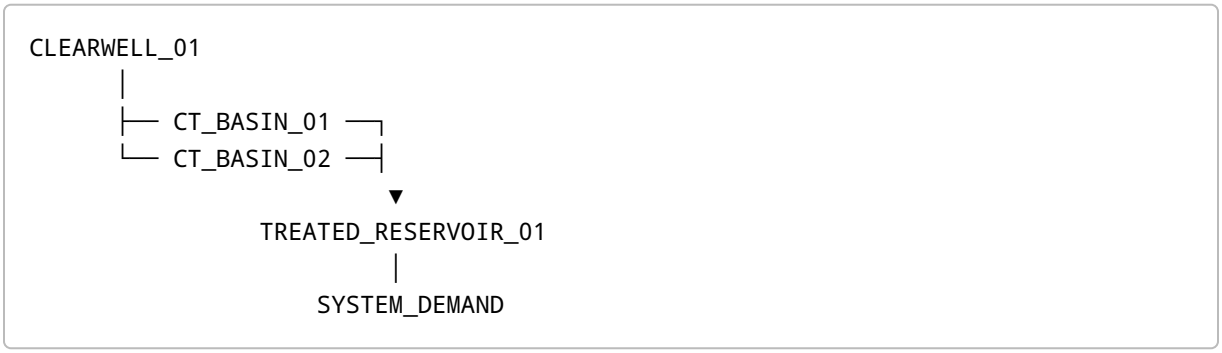
8. Phase 4b — Contact basins and treated water

Batch 12 — Author the detailed Phase 4b specification

Goal

Define the remainder of the hydraulic proof of concept without introducing unnecessary controller architecture.

Topology



Decisions

1. Create a new plant: `config/plants/phase4b_complete_train/`.
2. Preserve the Phase 4a plant as a regression fixture.
3. Model both CT basins and treated storage as generic `StorageUnit`s.
4. Use FlowSolver for parallel CT distribution.
5. Define one supervisory loop using existing controller capability.
6. Recommended loop:
7. Treated-reservoir level is the PV.

8. A shared clearwell-to-CT feed actuator is the target.
9. Low treated storage increases upstream feed.
10. Do not implement controller-to-controller cascade messaging unless a specific operating requirement proves the direct supervisory loop insufficient.
11. Exclude:
12. Regulatory CT calculations.
13. Chlorine residual.
14. Baffling factors.
15. Disinfection chemistry.
16. Pump curves.
17. Recycle paths.

Exit criteria

A binding Phase 4b implementation plan exists with control direction, capacities, alarms, tests, and exact topology.

Batch 13 — Implement and verify the complete hydraulic train

Work

1. Add two CT basin StorageUnits.
2. Add clearwell-to-CT links.
3. Add CT-to-treated-reservoir links.
4. Add treated-water storage.
5. Add system-demand boundary.
6. Add service-state controls for CT basins.
7. Add the treated-storage supervisory controller.
8. Add high/low alarms for:
9. Clearwell.
10. CT basins only where operationally required.
11. Treated-water storage.
12. Extend snapshots and presentation using existing generic paths.
13. Add low-poly CT basin and treated-storage visuals.
14. Add unit selection and service controls.

Tests

- Equal and capacity-limited CT distribution.
- One CT basin out of service.
- Both CT basins unavailable.

- Treated demand step.
- Treated-reservoir level control.
- Clearwell/treated-storage interaction.
- Full-train mass balance.
- Full-train no-negative-storage.
- Replay.
- Registration-order independence.
- 100,000-tick full-train soak.
- Headless/visual parity.
- All earlier plant configurations unchanged.

Exit criteria

The complete hydraulic process train is playable and verified.

---

9. Batch 14 — Complete the proof of concept

Goal

Close the gap between a complete hydraulic train and the project's published completion criteria.

The project scope requires pause, acceleration, reset, stepping, unit selection, alarms, service states, and extended mass-conserving operation.

Remaining application work

1. Add a real reset path.

2. Rebuild the plant from its original configuration.

3. Reset the clock, command queue, events, alarms, controllers, ledger, and snapshot.
4. Do not mutate an old snapshot back into production state.
5. Add a Reset control to the time-controls UI.
6. Verify pause, play, step, and all configured speed levels.
7. Ensure every process unit is selectable.
8. Ensure every supported service-state transition is available.
9. Ensure active alarms are visible and clear correctly.
10. Verify the alarm view and asset panel do not mutate snapshots.
11. Run a final full-train soak and churn suite.
12. Audit asset licenses and third-party notices.
13. Update:
    - README.
    - Roadmap.
    - Known limitations.
    - Architecture.

    - Configuration reference.
    - Controls.
    - Topology.
    - Changelog.
14. Remove completed dispatch documents from the active workflow or mark them complete.
15. Produce a tagged proof-of-concept release.
16. Optionally create a web or desktop build only after the test gate passes.

Reset is not currently exposed by either `SimulationHost` or the time-controls controller.

Final acceptance matrix

The POC is complete only when:

- Entire train is represented.
- Every storage level is numerically and visually correct.
- Valves affect flow.
- Basins, filters, and CT units can be taken out of service.
- Flow redistributes through five basins, twelve filters, and two CT basins.
- Spills and drains work.
- Manual and automatic modes work.
- Alarms activate and clear.
- Pause, step, speed, and reset work.
- Every unit is selectable.
- Long runs conserve mass.
- CI is green.
- Documentation matches production.

---

10. Blocker register

Blocker	Severity	Blocks	Resolution
Full tests not actually executed	Hard	All downstream batches	Complete Batch 0
Documentation does not match production	Hard for agent work	Safe audit implementation	Batch 1
Duck-typed gravity-head lookup	Hard for trusted gravity foundation	Phase 3 closure and Phase 4 gravity expansion	Batch 2.1
Unsupported `COMMANDED` and reverse-flow paths	Hard for honest contracts	Phase 3 closure	Batches 2.2–2.3

Blocker	Severity	Blocks	Resolution
Startup and alarms not configuration-driven	Hard for Phase 3 exit	Batch 6	Batch 3
Hardcoded CI script count	Process blocker	Reliable continued expansion	Batch 4
Controller semantics disagreement	Hard for new control design	Phase 4a controller plan	Batch 5A
Real Phase 3 elevation data absent	Soft	Calibrated realism	Human Track H1 / Batch 5B
Phase 3 gate open	Hard	Active Phase 4a plan and implementation	Batch 6
Filter and clearwell design basis absent	Hard unless placeholders are explicitly approved	Phase 4a gravity configuration	Human Track H2
Phase 4a gate open	Hard	Phase 4b	Batch 11
CT and treated-water design basis absent	Hard	Phase 4b configuration	Human Track H3

11. Safe parallel execution map

During audit closure

Lane A — Core implementation

- Batch 2.
- Batch 3.
- Batch 4.

Run these sequentially on the critical path.

Lane B — Documentation preparation

May inventory stale documentation in parallel, but final edits should be rebased after the related production changes so the docs describe the landed code.

Lane C — Plant-owner data gathering

- Phase 3 hydraulic profile.
- Filter and clearwell data.
- CT and treated-water data.

- Site-plan and presentation coordinates.

This lane can run throughout audit closure.

During Phase 4a

After topology IDs are frozen in Batch 8:

- Batch 9 control/service work.
- Batch 10 presentation/UI work.

These can run in parallel with separate file ownership.

During Phase 4b

After topology and control contracts are frozen:

- Hydraulic/configuration work.
- Presentation-map and asset placement.
- Scenario and parity test preparation.

Final integration and gate verification remain serial.

---

12. Explicitly deferred work

Do not add any of the following during these batches:

- Backwash.
- Filter-to-waste.
- Cyclic hydraulic topology.
- Reverse flow.
- `COMMANDED` flow.
- Pump curves.
- Detailed hydraulic grade-line calculations.
- Chemistry or dosing.
- Turbidity removal.
- Chlorine residual.
- Regulatory CT.
- Filter-media condition.
- Scenario framework.
- Cyber-physical scenarios.
- Historian infrastructure.
- Trend-buffer framework.
- MQTT or external APIs.
- Save-game architecture.

- Multiplayer or scoring.
- General interlock/permissive frameworks.
- Process-unit inheritance hierarchies.
- Asset polish unrelated to the next playable milestone.

These capabilities already have appropriate revisit triggers in the roadmap and should remain parked until a concrete failing case or committed milestone requires them.

---

13. Recommended work-package naming

```
WP4.2   Documentation and active-contract reconciliation
WP4.2a  Typed hydraulic-head contract
WP4.3   Remove COMMANDED mode
WP4.4   Remove reverse-flow support
WP4.5   Configured startup and alarm wiring
WP4.6   Derived CI test discovery
WP4.6a  Controller-semantics characterization
WP4.1b  Gravity-basis reconciliation, when data is available
WP4.7   Phase 3 and WP4.1 closure

WP5.0   Phase 4a specification
WP5.1   Twelve-filter and clearwell configuration
WP5.2   Filter service and clearwell control
WP5.3   Phase 4a presentation
WP5.4   Phase 4a verification

WP6.0   Phase 4b specification
WP6.1   CT basins and treated-water hydraulics
WP6.2   Supervisory control and operations
WP6.3   Full-train presentation
WP6.4   Full POC verification and release
```

The numbering deliberately preserves existing WP4 history and avoids renumbering completed work.